Experiment No. 6

Aim

To convert selected decimal numbers into binary form, append parity bits for even or odd parity, and understand simple error detection in digital data transmission.

Materials Required

- Python
- Python IDLE / VS Code
- Pen
- Notebook

Theory

Binary is a number system that uses only two digits, **0 and 1**. Computers and digital systems use binary to represent and process data.

Decimal to Binary Conversion

A decimal number can be converted into binary using the **division-by-2 method**. The remainders are recorded and read from bottom to top.

Example: Decimal 42

Division	Remainder
42 ÷ 2 = 21	0
21 ÷ 2 = 10	1
10 ÷ 2 = 5	0
5 ÷ 2 = 2	1
2 ÷ 2 = 1	0
1 ÷ 2 = 0	1

Therefore,

$42_{10} = 101010_2$

This is the same worked example used in the Experiment 6 material.

Parity Bit

A **parity bit** is an additional bit added to binary data to help detect errors during transmission.

- **Even Parity:** Total number of 1s must be even.
- **Odd Parity:** Total number of 1s must be odd.

For **101010**, there are three 1s. For even parity, we append **1**.

101010 + 1 = 1010101

Now there are four 1s, so the even parity condition is satisfied.

Python Code

```
def simulate_data_transmission(decimal_data):
    print("========================================")
    print("   TRANSMISSION PREPARATION (STEP 1)    ")
    print("========================================")

    # 1. Convert decimal to binary (removing the '0b' prefix python
adds)
    binary_data = bin(decimal_data)[2:]

    # 2. Count the number of '1's in the binary string
    ones_count = binary_data.count('1')

    print(f"Original Decimal Data : {decimal_data}")
    print(f"Binary Representation : {binary_data}")
    print(f"Count of '1's         : {ones_count}")
    print("\n")

    print("========================================")
    print("      EVEN PARITY SYSTEM (STEP 2)       ")
    print("========================================")
    # Rule: Total number of 1s (including parity bit) must be EVEN.
    # If the count is already even, append '0'. If odd, append '1'.
    if ones_count % 2 == 0:
        even_parity_bit = '0'
    else:
        even_parity_bit = '1'

    even_transmission_frame = binary_data + even_parity_bit

    print(f"Parity Bit Appended   : {even_parity_bit}")
    print(f"Final Data Frame Sent : {even_transmission_frame}")
    print("\n")

    print("========================================")
    print("       ODD PARITY SYSTEM (STEP 3)       ")
    print("========================================")
    # Rule: Total number of 1s (including parity bit) must be ODD.
    # If the count is even, append '1'. If it is already odd, append
'0'.
    if ones_count % 2 == 0:
```

```
        odd_parity_bit = '1'
    else:
        odd_parity_bit = '0'

    odd_transmission_frame = binary_data + odd_parity_bit

    print(f"Parity Bit Appended   : {odd_parity_bit}")
    print(f"Final Data Frame Sent : {odd_transmission_frame}")
    print("========================================\n")

# --- Run the Experiment ---
# You can change the number 42 to any decimal number to test different
outputs!
simulate_data_transmission(50)
```

Example Output

```
========================================
   TRANSMISSION PREPARATION (STEP 1)
========================================
Original Decimal Data : 50
Binary Representation : 110010
Count of '1's         : 3

========================================
      EVEN PARITY SYSTEM (STEP 2)
========================================
Parity Bit Appended   : 1
Final Data Frame Sent : 1100101

========================================
       ODD PARITY SYSTEM (STEP 3)
========================================
Parity Bit Appended   : 0
Final Data Frame Sent : 1100100

========================================
```

Observation

Decimal	Binary	No. of 1s	Even Parity Bit	Final Data
10	1010	2	0	10100
15	1111	4	0	11110
42	101010	3	1	1010101

Error Detection Example

Transmitted data:

1010101

If one bit changes during transmission:

1010101 → 0010101

The received data contains **three 1s**. Since even parity requires an even number of 1s, an **error is detected**. The supplied Experiment 6 material uses this same example of a corrupted bit.

Applications

Ancient Applications

- Transmission of oral traditions.
- Vedic chanting and recitation.
- Maintaining rhythmic structures.

Modern Applications

- Binary coding in computers.
- Digital communication.
- Data storage.
- Error detection during data transmission.

Result

The decimal number was successfully converted into binary form, the appropriate parity bit was added, and a simple transmission error was successfully detected.

Conclusion

The experiment demonstrated decimal-to-binary conversion and the use of parity bits for simple error detection. Binary representation and parity are important concepts in modern digital communication and data systems.